Time Series Forecasting — Versi 2: Foundation Model (Chronos-2)

Saham: UNTR.JK | **Model**: amazon/chronos-2 (120M params) | **Pendekatan**: Zero-Shot Forecasting

Versi ini menggunakan **pretrained foundation model** dari Amazon — tanpa training dari awal. Model hanya menerima historical data dan langsung menghasilkan forecast probabilistik.

Aspek	Versi 1 (Klasik/ML)	Versi 2 (Chronos-2)
Training	Dari awal	Tidak perlu
Feature Engineering	Perlu	Tidak perlu
Inferensi	Cepat	Membutuhkan GPU
Interpretabilitas	Tinggi	Rendah (black box)
Akurasi (zero-shot)	Baseline	State-of-the-art

```
Mulai coding atau buat kode dengan AI.
```

```
# Install dependencies (otomatis jika di Google Colab)
import subprocess, sys
IN_COLAB = 'google.colab' in sys.modules
if IN_COLAB:
    print("Running di Google Colab - GPU T4 tersedia!")
    subprocess.run([sys.executable, '-m', 'pip', 'install',
                    'yfinance', 'chronos-forecasting>=2.0', '-q'], check=True)
    print("Install selesai!")
else:
    print("Running secara lokal - pastikan requirements_v2.txt sudah diinstall")
```

```
Running di Google Colab - GPU T4 tersedia!
Install selesai!
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.dates as mdates
import seaborn as sns
import warnings
warnings.filterwarnings('ignore')
from datetime import datetime, timedelta
import yfinance as yf
import torch
from sklearn.metrics import mean_absolute_error, mean_squared_error

try:
    plt.style.use('seaborn-v0_8-whitegrid')
except:
    plt.style.use('ggplot')

TICKER = 'UNTR.JK'
device = 'cuda' if torch.cuda.is_available() else 'cpu'
print(f"Device  : {device}")
if device == 'cuda':
    print(f"GPU     : {torch.cuda.get_device_name(0)}")
    print(f"VRAM    : {torch.cuda.get_device_properties(0).total_memory/1e9:.1f} GB")
print("Libraries OK")
```

```
Device  : cuda
GPU     : Tesla T4
VRAM    : 15.6 GB
Libraries OK
```

1. Data Acquisition

```
end = datetime.today()
start = end - timedelta(days=365*10)
print(f"Mengunduh {TICKER}: {start.date()} to {end.date()}")
raw = yf.download(TICKER, start=start, end=end, auto_adjust=True, progress=False)
df = raw[['Close','Volume']].copy().dropna()
df.index = pd.DatetimeIndex(df.index)
close = df['Close'].squeeze()
print(f"Total: {len(df)} hari trading | {df.index[0].date()} to {df.index[-1].date()}")
df.tail()
```

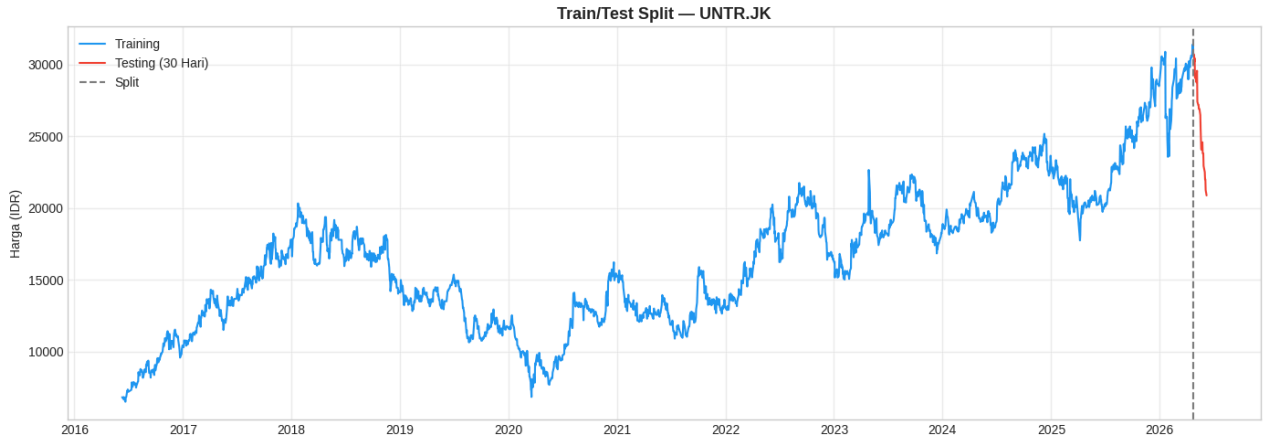
```
Mengunduh UNTR.JK: 2016-06-10 to 2026-06-08
Total: 2457 hari trading | 2016-06-10 to 2026-06-08
```

Price	Close	Volume
Ticker	UNTR.JK	UNTR.JK
Date		
2026-06-02	22500.0	8257600
2026-06-03	22000.0	7258900
2026-06-04	22000.0	6039600
2026-06-05	21250.0	5219300
2026-06-08	20875.0	5393000

2. Train / Test Split (sama seperti Versi 1)

```
# Test period: 30 trading days (standar portfolio DS - cukup untuk semua model)
N_TEST_DAYS = 30
split_date = df.index[-N_TEST_DAYS]
train_close = close[:split_date]
test_close  = close[split_date:]
n_test = len(test_close)
print(f"Train: {train_close.index[0].date()} -> {train_close.index[-1].date()} ({len(train_close)} hari)")
print(f"Test : {test_close.index[0].date()} -> {test_close.index[-1].date()} ({n_test} hari)")

fig, ax = plt.subplots(figsize=(14,5))
ax.plot(train_close.index, train_close.values, color='#2196F3', lw=1.5, label='Training')
ax.plot(test_close.index, test_close.values, color='#F44336', lw=1.5, label=f'Testing ({n_test} Hari)')
ax.axvline(split_date, color='gray', ls='--', label='Split')
ax.set_title(f'Train/Test Split — {TICKER}', fontsize=13, fontweight='bold')
ax.legend(); ax.grid(alpha=0.3); ax.set_ylabel('Harga (IDR)')
plt.tight_layout(); plt.savefig('split_chronos.png', dpi=150, bbox_inches='tight'); plt.show()
```

```
Train: 2016-06-10 -> 2026-04-24 (2428 hari)
Test : 2026-04-24 -> 2026-06-08 (30 hari)
```



3. Load Chronos-2 Foundation Model

```
# Install jika belum ada:
# pip install "chronos-forecasting>=2.0" torch

from chronos import BaseChronosPipeline

print(f"Memuat amazon/chronos-2 ke {device}...")
print("(Download ~500MB saat pertama kali, akan ter-cache setelahnya)")

pipeline = BaseChronosPipeline.from_pretrained(
    "amazon/chronos-2",
```

```
    device_map=device,
    torch_dtype=torch.float32
)
print("Model Chronos-2 berhasil dimuat!")
```

```
Memuat amazon/chronos-2 ke cuda...
(Download ~500MB saat pertama kali, akan ter-cache setelahnya)
Model Chronos-2 berhasil dimuat!
```

4. Zero-Shot Forecasting

```
context_series = torch.tensor(train_close.values, dtype=torch.float32)
print(f"Context window: {len(context_series)} time steps")
print(f"Forecasting: {n_test} steps ke depan")

# Quantile forecast (probabilistik)
quantile_levels = [0.1, 0.25, 0.5, 0.75, 0.9]

print("\nMenjalankan zero-shot forecasting...")
# Chronos-2 expects a list of tensors for the first positional argument `inputs`
forecast = pipeline.predict(
    [context_series],         # Passed as inputs list
    prediction_length=n_test
)
# forecast shape: (1, num_samples, prediction_length)
print(f"Forecast shape: {forecast[0].shape}")
```

```
Context window: 2428 time steps
Forecasting: 30 steps ke depan

Menjalankan zero-shot forecasting...
Forecast shape: torch.Size([1, 21, 30])
```

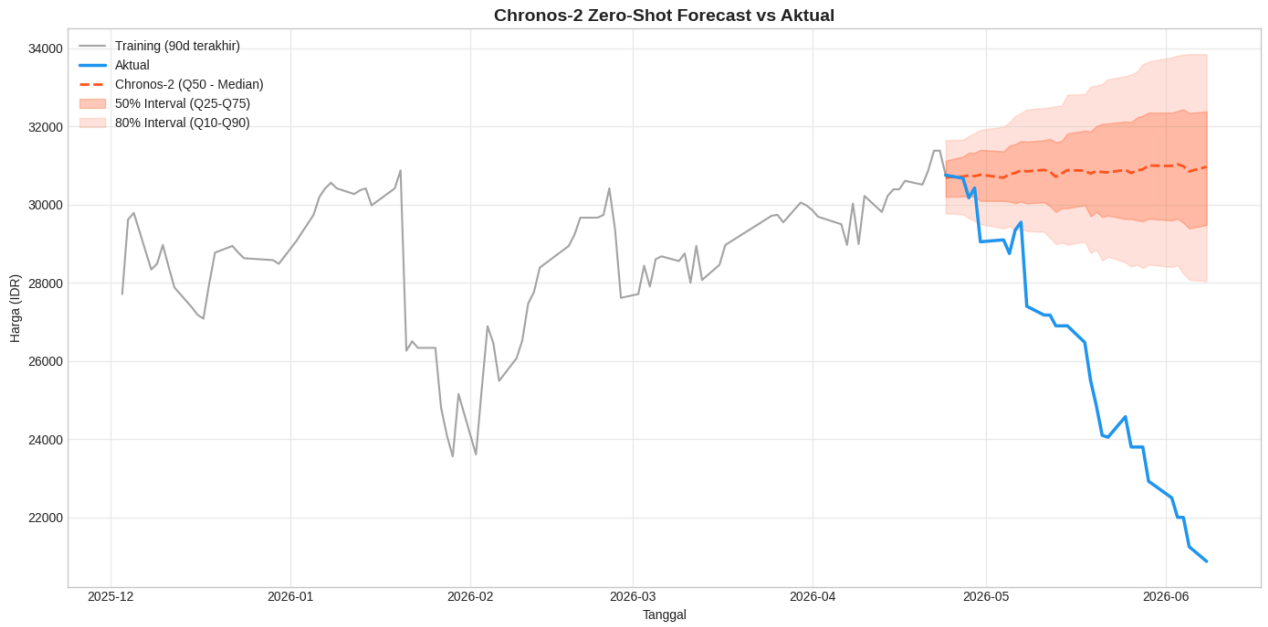
```
# Hitung quantiles dari samples
fc_samples = forecast[0][0].numpy()  # (num_samples, n_test)
fc_q10 = np.quantile(fc_samples, 0.10, axis=0)
fc_q25 = np.quantile(fc_samples, 0.25, axis=0)
fc_q50 = np.quantile(fc_samples, 0.50, axis=0)  # median = point forecast
fc_q75 = np.quantile(fc_samples, 0.75, axis=0)
fc_q90 = np.quantile(fc_samples, 0.90, axis=0)

print(f"Prediksi pertama (Q50): {fc_q50[0]:,.0f}")
print(f"Prediksi terakhir (Q50): {fc_q50[-1]:,.0f}")
print(f"Aktual pertama: {test_close.values[0]:,.0f}")
print(f"Aktual terakhir: {test_close.values[-1]:,.0f}")
```

```
Prediksi pertama (Q50): 30,695
Prediksi terakhir (Q50): 30,968
Aktual pertama: 30,754
Aktual terakhir: 20,875
```

5. Visualisasi Forecast dengan Prediction Interval

```
fig, ax = plt.subplots(figsize=(14,7))

# Plot training (90 hari terakhir)
ax.plot(train_close.index[-90:], train_close.values[-90:],
        color='gray', lw=1.5, alpha=0.7, label='Training (90d terakhir)')

# Plot aktual (test)
ax.plot(test_close.index, test_close.values,
        color='#2196F3', lw=2.5, label='Aktual', zorder=10)

# Plot prediksi median
ax.plot(test_close.index[:n_test], fc_q50,
        color='#FF5722', lw=2, ls='--', label='Chronos-2 (Q50 - Median)')

# Prediction intervals
ax.fill_between(test_close.index[:n_test], fc_q25, fc_q75,
                alpha=0.3, color='#FF5722', label='50% Interval (Q25-Q75)')
ax.fill_between(test_close.index[:n_test], fc_q10, fc_q90,
                alpha=0.15, color='#FF5722', label='80% Interval (Q10-Q90)')

ax.set_title('Chronos-2 Zero-Shot Forecast vs Aktual', fontsize=14, fontweight='bold')
ax.set_xlabel('Tanggal'); ax.set_ylabel('Harga (IDR)')
ax.legend(loc='upper left'); ax.grid(alpha=0.3)
```

```
plt.tight_layout(); plt.savefig('chronos_forecast.png', dpi=150, bbox_inches='tight'); plt.show()
```



6. Evaluasi Metrik

```
y_true = test_close.values[:n_test]
y_pred = fc_q50[:n_test]

mae  = mean_absolute_error(y_true, y_pred)
rmse = np.sqrt(mean_squared_error(y_true, y_pred))
mape = np.mean(np.abs((y_true - y_pred) / y_true)) * 100

print("="*40)
print("EVALUASI CHRONOS-2 (Zero-Shot)")
print("="*40)
print(f"MAE  : {mae:,.2f}")
print(f"RMSE : {rmse:,.2f}")
print(f"MAPE : {mape:.2f}%")
print("="*40)

chronos_result = {'Model':'Chronos-2 (Zero-Shot)','MAE':round(mae,2),'RMSE':round(rmse,2),'MAPE':round(mape,2)}
```

```
========================================
EVALUASI CHRONOS-2 (Zero-Shot)
========================================
MAE  : 4,753.62
RMSE : 5,646.31
MAPE : 19.84%
========================================
```

7. Perbandingan dengan Model Klasik (Versi 1)

```
# Load hasil evaluasi Versi 1 (jika sudah dijalankan)
import os

v1_results_path = 'hasil_evaluasi_v1.csv'
if not os.path.exists(v1_results_path):
    v1_results_path = '../hasil_evaluasi_v1.csv'

if os.path.exists(v1_results_path):
```

```
    df_v1 = pd.read_csv(v1_results_path, index_col=0)
    df_chronos = pd.DataFrame([chronos_result]).set_index('Model')
    df_all = pd.concat([df_v1, df_chronos]).sort_values('MAPE')
    print("Perbandingan SEMUA Model (Versi 1 + Chronos-2):")
    print(df_all.to_string())
    df_all.to_csv('hasil_evaluasi_final.csv')
    print("\nTersimpan ke hasil_evaluasi_final.csv")
else:
    # Jika V1 belum dijalankan, tampilkan hasil Chronos saja
    print("Catatan: Jalankan V1 dulu untuk mendapatkan perbandingan lengkap")
    df_all = pd.DataFrame([chronos_result]).set_index('Model')
    print(df_all.to_string())
```

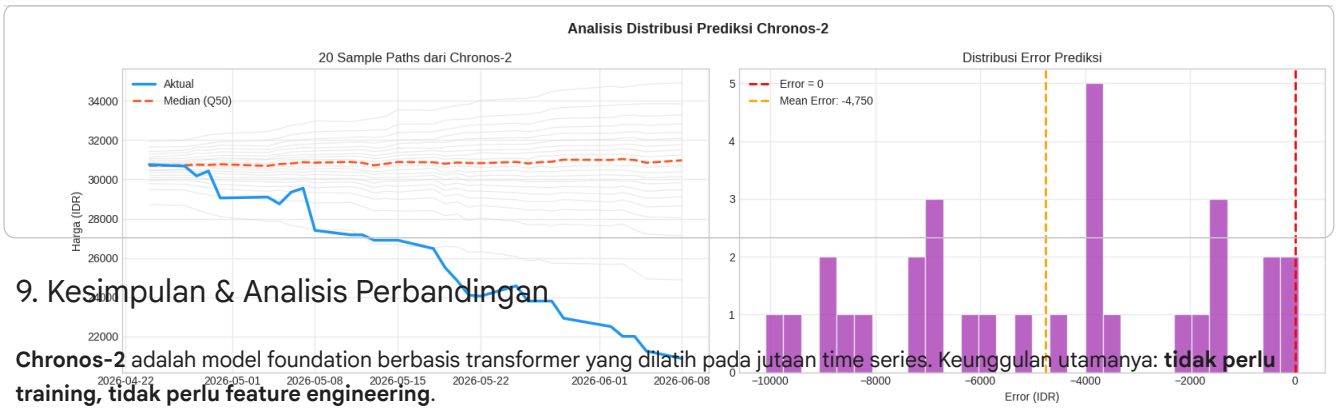
```
Catatan: Jalankan V1 dulu untuk mendapatkan perbandingan lengkap
                            MAE     RMSE   MAPE
Model
Chronos-2 (Zero-Shot)  4753.62  5646.31  19.84
```

```
# Visualisasi perbandingan MAPE
if len(df_all) > 1:
    df_sorted = df_all.sort_values('MAPE')
    colors = ['#FF5722' if 'Chronos' in idx else '#2196F3' for idx in df_sorted.index]
    fig, ax = plt.subplots(figsize=(10, 6))
    bars = ax.barh(df_sorted.index, df_sorted['MAPE'], color=colors, alpha=0.85, height=0.6)
    ax.bar_label(bars, fmt='%.2f%%', padding=5, fontsize=10)
    ax.set_xlabel('MAPE (%)'); ax.invert_yaxis()
    ax.set_title('Perbandingan MAPE: Klasik vs Chronos-2', fontsize=13, fontweight='bold')
    from matplotlib.patches import Patch
    ax.legend(handles=[Patch(color='#FF5722', label='Chronos-2'), Patch(color='#2196F3', label='Model Klasik/ML')])
    ax.grid(axis='x', alpha=0.3)
    plt.tight_layout(); plt.savefig('comparison_chronos_vs_classical.png', dpi=150, bbox_inches='tight'); plt.show()
else:
    print("Jalankan V1 notebook terlebih dahulu untuk perbandingan lengkap.")
```

```
Jalankan V1 notebook terlebih dahulu untuk perbandingan lengkap.
```

8. Analisis Distribusi Prediksi Chronos-2

```
# Sample paths dari Chronos
fig, axes = plt.subplots(1, 2, figsize=(16, 5))
fig.suptitle('Analisis Distribusi Prediksi Chronos-2', fontsize=13, fontweight='bold')

# Plot beberapa sample paths
ax = axes[0]
ax.plot(test_close.index, test_close.values, color='#2196F3', lw=2.5, label='Aktual', zorder=10)
for i in range(min(20, fc_samples.shape[0])):
    ax.plot(test_close.index[:n_test], fc_samples[i], color='gray', alpha=0.15, lw=0.8)
ax.plot(test_close.index[:n_test], fc_q50, color='#FF5722', lw=2, ls='--', label='Median (Q50)')
ax.set_title('20 Sample Paths dari Chronos-2'); ax.legend(); ax.grid(alpha=0.3); ax.set_ylabel('Harga (IDR)')

# Distribusi error
ax2 = axes[1]
errors = y_true[:n_test] - fc_q50[:n_test]
ax2.hist(errors, bins=30, color='#9C27B0', alpha=0.7, edgecolor='white')
ax2.axvline(0, color='red', ls='--', lw=2, label='Error = 0')
ax2.axvline(np.mean(errors), color='orange', ls='--', lw=2, label=f'Mean Error: {np.mean(errors):,.0f}')
ax2.set_title('Distribusi Error Prediksi'); ax2.set_xlabel('Error (IDR)'); ax2.legend(); ax2.grid(alpha=0.3)

plt.tight_layout(); plt.savefig('chronos_distribution.png', dpi=150, bbox_inches='tight'); plt.show()
```



9. Kesimpulan & Analisis Perbandingan

Chronos-2 adalah model foundation berbasis transformer yang dilatih pada jutaan time series. Keunggulan utamanya: **tidak perlu training, tidak perlu feature engineering**.

Kelebihan Chronos-2

- Zero-shot: langsung predict tanpa training
- Mengeluarkan prediksi probabilistik (quantile intervals)
- Robust terhadap berbagai jenis data time series
- State-of-the-art pada benchmark GIFT-Eval dan Chronos Benchmark II

Kekurangan Chronos-2

- Membutuhkan GPU untuk inferensi yang efisien
- Black-box: sulit diinterpretasikan
- Tidak memanfaatkan domain knowledge (misalnya berita, fundamental saham)
- Untuk dataset spesifik, model klasik yang dilatih ulang bisa lebih unggul

Kesimpulan Akhir Proyek

Pendekatan	Kelebihan	Cocok untuk
Klasik (MA/ARIMA)	Cepat, explainable	Baseline, laporan sederhana
ML (XGBoost/LSTM)	Akurasi tinggi	Dataset besar, non-linear
Foundation Model (Chronos-2)	Zero-shot, probabilistik	Eksplorasi cepat, multiple series

> Tidak ada model yang selalu terbaik. Pilihan tergantung pada kebutuhan, data, dan resources yang tersedia.